

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 01 -

FUNDAMENTOS DE ESCALABILIDADE MODERNA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	10
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMANIP

O QUE VEM POR AÍ?

Imagine que sua empresa acabou de lançar um aplicativo inovador. Após uma campanha de marketing bem-sucedida, ele “viraliza” e milhões de usuários tentam acessá-lo simultaneamente. O que deveria ser um momento de sucesso da companhia se transforma em um pesadelo: o sistema não aguenta o alto número de acessos e sai do ar, gerando frustração nos clientes e perda de receita. Esse cenário, mais comum do que se imagina, é o resultado direto de uma falha de escala. A capacidade de crescer e lidar com o aumento da demanda não é um luxo, mas uma necessidade fundamental no mundo digital.

Nesta aula, vamos mergulhar nos conceitos essenciais que sustentam os sistemas modernos e resilientes. Você aprenderá a diferenciar o que é, de fato, escalar, de simplesmente otimizar ou melhorar o desempenho. Exploraremos os trade-offs inevitáveis entre performance, custo e complexidade, e desvendaremos os métodos e ferramentas que gigantes da tecnologia usam para garantir que suas portas digitais estejam sempre abertas, não importa o quão grande seja a multidão do lado de fora. Prepare-se para construir a base que o impedirá de se tornar o próximo "caso de falha" da internet.

HANDS ON

Nas videoaulas que acompanham este material, nosso foco será na construção de uma base conceitual sólida, um verdadeiro "hands on" de fundamentos. Em vez de nos aprofundarmos em linhas de código complexas, vamos visualizar e desconstruir as ideias que formam o alicerce da escalabilidade. A meta é que você não apenas memorize os termos, mas que internalize o "porquê" por trás de cada decisão arquitetural, um passo crucial para quem deseja liderar projetos de tecnologia.

Vamos começar usando analogias e diagramas para ilustrar a diferença crucial entre escalar um sistema, otimizá-lo e simplesmente medir seu desempenho. Em seguida, partiremos para uma exploração visual dos **dois métodos primários de escala**: o **vertical** e o **horizontal**. Você verá, na prática, as vantagens e desvantagens de cada um, entendendo quando faz sentido "comprar um servidor mais potente" e quando a melhor solução é "adicionar mais servidores à frota".

Por fim, daremos uma olhada em como esses conceitos se materializam em ferramentas reais do mercado. Analisaremos de forma conceitual como funcionam os painéis de controle de soluções como o AWS Auto Scaling Groups (ASG), entendendo o que significam termos como "elasticidade" e "autoscaling". Ao final desta aula prática-conceitual, você terá a clareza necessária para identificar as estratégias de escalabilidade mais adequadas para os diferentes cenários. Ah, e não se esqueça de revisar os vídeos sempre que precisar reforçar esses conceitos!

SAIBA MAIS

Para construir sistemas que suportem o crescimento exponencial do mundo digital, é preciso, antes de tudo, dominar o vocabulário. Muitas vezes, os termos "desempenho", "otimização" e "escalabilidade" são usados de forma intercambiável, mas representam conceitos distintos que exigem soluções diferentes.

Desempenho ou performance, refere-se à velocidade com que uma única tarefa é executada, ou seja, a latência de uma operação. Pense em um caixa de supermercado: seu desempenho é medido por quantos itens ele consegue escanear por minuto. Já a otimização é o ato de melhorar esse desempenho sem adicionar novos recursos; por exemplo, ensinando ao caixa uma técnica para escanear produtos 20% mais rápido ou reorganizando o layout dos produtos para facilitar seu trabalho.

A escalabilidade, por sua vez, é a capacidade de um sistema de aumentar sua produtividade (throughput) de forma proporcional aos recursos adicionados. No nosso exemplo, quando a fila de clientes se torna gigantesca, não adianta ter o caixa mais rápido do mundo; a solução é escalar, ou seja, abrir novos caixas para atender à demanda.

Essa decisão de como escalar nos leva a um dos dilemas centrais da arquitetura de software: o **triângulo dos trade-offs**, formado pelos vértices de **performance, custo e complexidade**. É praticamente impossível maximizar os três simultaneamente. Se aprofundar em um deles geralmente implica sacrificar outro. Por exemplo, para obter a máxima performance de um banco de dados, pode-se usar hardware de ponta e várias camadas de cache, o que eleva drasticamente o custo e a complexidade de manutenção. Da mesma forma, uma arquitetura de microsserviços pode oferecer um custo menor por requisição em larga escala, mas introduz uma complexidade operacional imensa em comparação com um sistema monolítico. Como profissional de tecnologia, seu papel será constantemente navegar por esses trade-offs, buscando o equilíbrio que melhor atende aos requisitos técnicos e de negócio do projeto.

Uma vez compreendido o "o quê" e o "porquê", passamos ao "como". Existem **dois métodos primários para escalar um sistema**: a **escala vertical** e a **escala horizontal**.

A **escala vertical** (ou scaling up) consiste em aumentar a capacidade de um único servidor, adicionando mais poder de fogo: mais núcleos de CPU, mais memória RAM ou discos mais rápidos. É como fazer um upgrade no seu computador pessoal, trocando componentes por versões mais potentes. A principal vantagem é a simplicidade, pois a aplicação geralmente não precisa de alterações de código para se beneficiar dos novos recursos. No entanto, essa abordagem tem desvantagens significativas. Primeiro, existe um limite físico e financeiro — chega um ponto em que o próximo upgrade se torna proibitivamente caro ou simplesmente não existe. Segundo, ela geralmente exige um tempo de inatividade (downtime) para a realização do upgrade. E, o mais crítico, ela cria um ponto único de falha (Single Point of Failure): se aquele super-servidor parar, todo o sistema para com ele.

Por outro lado, a **escala horizontal** (ou scaling out) envolve adicionar mais servidores para distribuir a carga de trabalho. Em vez de um único servidor superpotente, temos uma "frota" de servidores menores e, geralmente, mais baratos. Essa abordagem oferece uma capacidade de escala teoricamente ilimitada e uma resiliência muito maior, pois a falha de um servidor não compromete o sistema como um todo; o tráfego é simplesmente redirecionado para as outras instâncias saudáveis. Contudo, ela exige que a aplicação seja projetada para esse ambiente distribuído (idealmente, sendo "stateless") e introduz uma maior complexidade operacional, como a necessidade de um balanceador de carga (Load Balancer) para distribuir o tráfego de forma inteligente.

Horizontal Scaling vs. Vertical Scaling

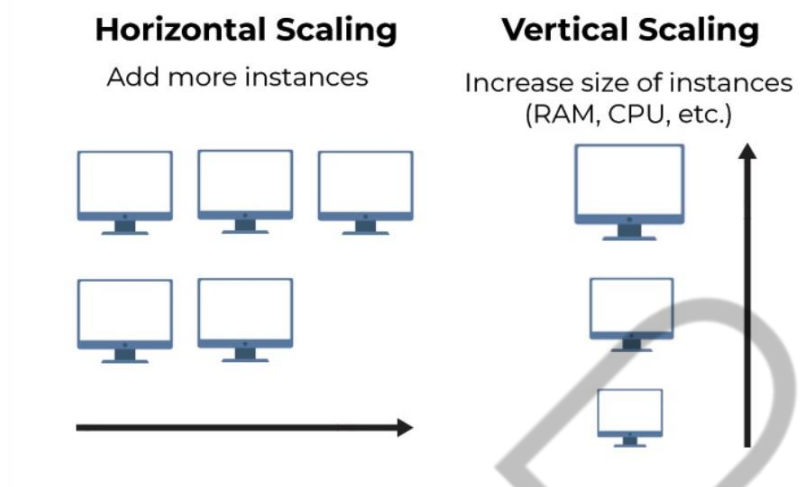


Figura 1 - Escala Vertical vs Escala Horizontal
Fonte: <https://www.nops.io/blog/horizontal-vs-vertical-scaling/> (2025)

É no contexto da escala horizontal que surgem os conceitos de elasticidade e autoscaling, que são o coração da computação em nuvem moderna.

A **elasticidade** é a capacidade de um sistema de se adaptar dinamicamente à carga de trabalho, crescendo ou encolhendo seus recursos conforme a necessidade.

O **autoscaling** é o mecanismo que implementa essa elasticidade de forma automatizada. Em vez de uma equipe de plantão adicionando servidores manualmente durante um pico, o próprio sistema, com base em regras predefinidas, gerencia sua própria capacidade. Essas regras podem ser de vários tipos:

Scaling Baseado em Métricas: é o mais comum. Por exemplo: "Se a utilização média da CPU em toda a frota de servidores ultrapassar 70% por mais de 5 minutos, adicione um novo servidor. Se cair abaixo de 30% por 10 minutos, remova um servidor."

Scaling Agendado: é útil para demandas previsíveis. Por exemplo: "Todo dia útil, às 8h55, aumente o número de servidores para 10. Às 18h05, reduza para 2."

Scaling Preditivo: é uma abordagem mais avançada, que utiliza Machine Learning para analisar dados históricos e prever picos de demanda, ajustando a capacidade proativamente, antes que o pico aconteça.

Essa automação é a chave para a eficiência de custos na nuvem, permitindo pagar apenas pelos recursos efetivamente utilizados.

A relevância desses conceitos fica evidente quando analisamos os casos clássicos de falha de escala que marcaram a história da internet. Episódios como os sites de grandes varejistas saindo do ar durante a Black Friday ou o famoso "Fail Whale" (baleia da falha) do Twitter, não são apenas falhas técnicas; são eventos que causam prejuízos financeiros milionários e danos irreparáveis à reputação da marca. Tecnicamente, essas falhas geralmente ocorriam devido a gargalos em um componente que não podia ser escalado horizontalmente com facilidade, como um banco de dados relacional sobrecarregado ou um sistema de processamento de pagamentos que não suportava a concorrência. Esses exemplos nos ensinam que a capacidade de escalar não é apenas um requisito técnico, mas uma necessidade de negócio.



Figura 2 – “Fail Whale” (baleia da falha) nos primórdios do Twitter (atual “X”)
Fonte: Portal G1 (2025)

Os desafios para garantir a escalabilidade hoje são ainda maiores. Vivemos na era das arquiteturas de microsserviços, dos contêineres, dos sistemas orientados a eventos e dos picos sazonais imprevisíveis. Em vez de um único sistema monolítico, agora temos dezenas ou centenas de pequenos serviços, cada um com sua própria necessidade de escala. Essa granularidade é poderosa, pois permite escalar apenas o componente necessário (por exemplo, o serviço de carrinho de compras), mas também introduz desafios de latência de rede e observabilidade entre os serviços.

Para lidar com essa complexidade, surgiram ferramentas modernas. O Kubernetes, por exemplo, se tornou o padrão para orquestrar contêineres, enquanto

soluções como os Auto ScalingGroups (ASG) da AWS são fundamentais para a elasticidade na nuvem. Mais recentemente, ferramentas ainda mais inteligentes como o Karpenter, que provisiona recursos com base na necessidade real e com foco em custo, e o KEDA, que permite escalar sistemas com base em eventos (como o tamanho de uma fila), estão redefinindo os limites da eficiência.

Por fim, é crucial exercitar o pensamento crítico e fazer uma pergunta fundamental: “Quando não escalar?”

A escalabilidade é uma solução poderosa, mas não é a resposta para todos os problemas. Às vezes, um gargalo de performance pode ser resolvido com uma simples otimização no código ou uma query de banco de dados mais eficiente. Em outros casos, para uma aplicação com tráfego baixo e estável, como um sistema interno para 50 funcionários, o custo e a complexidade de manter um sistema de autoscaling seria um exagero. Saber discernir entre um problema de escala e um problema de otimização é uma marca de maturidade e sabedoria em engenharia de software.

MERCADO, CASES E TENDÊNCIAS

A escalabilidade continua sendo um dos temas mais relevantes e desafiadores no mercado de tecnologia, com empresas constantemente inovando para garantir disponibilidade e performance.

Case: A resiliência do iFood em dias de pico

O iFood, um dos maiores aplicativos de delivery da América Latina, é um excelente exemplo de escalabilidade na prática. Em dias de alta demanda, como em finais de campeonato de futebol ou dias chuvosos, a plataforma lida com um aumento massivo e repentino de pedidos. A empresa utiliza uma arquitetura de microsserviços na nuvem, com uso intensivo de autoscaling e sistemas de filas para desacoplar o processamento de pedidos, garantindo que o aplicativo permaneça responsivo para o usuário mesmo sob estresse extremo. O aprendizado contínuo com esses eventos permite que a equipe de engenharia refine constantemente suas estratégias de escalabilidade.

Tendência: FinOps e a Escalabilidade Consciente de Custos

Uma tendência crescente no mercado é a disciplina de FinOps (Financial Operations), que busca trazer responsabilidade financeira para os gastos na nuvem. Nesse contexto, a escalabilidade não é vista apenas pela ótica da performance, mas também pela da eficiência de custos. Ferramentas como o Karpenter ganham destaque por permitirem um autoscaling mais inteligente, provisionando os recursos computacionais exatos que uma aplicação precisa, no momento em que precisa, e utilizando os tipos de instância mais baratos disponíveis, otimizando a conta da nuvem sem sacrificar a performance.

Notícia: As lições dos grandes eventos on-line

Ano após ano, a venda de ingressos para grandes festivais de música ou eventos esportivos se torna um teste de fogo para as plataformas de e-commerce. Notícias sobre sites que saem do ar durante picos são comuns e servem como um lembrete público da importância de uma arquitetura robusta. Esses eventos forçam o mercado a desenvolver soluções cada vez mais sofisticadas de balanceamento de carga, proteção contra tráfego malicioso (bots) e estratégias de cache distribuído globalmente via CDNs.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula introdutória, estabelecemos as fundações do universo da escalabilidade. Você aprendeu a diferenciar claramente os conceitos de escalar, otimizar e desempenhar, entendendo como cada um contribui para a saúde de um sistema. Exploramos o trade-off fundamental entre performance, custo e complexidade, que guiará muitas de suas decisões futuras.

Apresentamos os dois métodos primários de escala, vertical e horizontal, e mergulhamos no conceito de elasticidade, que é a base da computação em nuvem moderna. Por fim, vimos como os desafios atuais, como microsserviços e picos sazonais, são enfrentados por ferramentas como Kubernetes, KEDA e Karpenter, e refletimos sobre a importante questão de quando a escalabilidade pode não ser a resposta certa.

REFERÊNCIAS

BEYER, B. et al. (Eds.). **The Site Reliability Workbook: Practical Ways to Implement SRE**. Sebastopol: O'Reilly Media, 2018.

KLEPPMANN, M. **Designing Data-Intensive Applications**. Sebastopol: O'Reilly Media, 2017.

NETFLIX TECHNOLOGY BLOG. **Netflix Technology Blog**. Disponível em: <https://netflixtechblog.com/>. Acesso em: 26 ago. 2025.

PALAVRAS-CHAVE

Escalabilidade. Escala Vertical. Escala Horizontal. Autoscaling. Elasticidade.

EMAP



POSTECH